In-Class Activity 6: Decision Trees, Random Forests, and Boosting

GEOL 599: Machine Learning

```
# If needed, install these packages first:
# install.packages(c("tidyverse", "tidymodels",
install.packages(c("rpart", "rpart.plot", "ranger", "xgboost", "patchwork"))
```

```
##
## The downloaded binary packages are in
##  /var/folders/1s/6n4p8nkj2vvcykj2dbz5794m0000gn/T//RtmpsQ2rrI/downloaded_packages
```

```
install.packages("rpart")
```

```
##
## The downloaded binary packages are in
##  /var/folders/1s/6n4p8nkj2vvcykj2dbz5794m0000gn/T//RtmpsQ2rrI/downloaded_packages
```

```
library(tidyverse)
library(tidymodels)
library(rpart)
library(rpart.plot)
library(ranger)
library(xgboost)
library(patchwork)

tidymodels_prefer()
set.seed(42)
```

Overview

In this activity, you will use the **Statlog Landsat Satellite** dataset to compare several tree-based machine-learning models:

- A single decision tree
- Random forest
- XGBoost

Each row represents a small 3 by 3 pixel neighborhood from Landsat satellite imagery. The predictors are spectral measurements from those pixels, and the target is the land-cover class of the central pixel.

Learning Goals

By the end of this activity, you should be able to:

- Explain why decision trees make **axis-aligned splits**.
- Visualize the limitations of a single decision tree boundary.
- Compare a single tree with ensemble tree methods.
- Explain the difference between bagging and boosting.
- Use train/test splits and cross-validation appropriately.

- Tune tree-based models without leaking information from the test set.
- Interpret model behavior using feature importance and permutation importance.

1. Load the Landsat Dataset

The dataset is the Statlog Landsat Satellite dataset from OpenML/UCI. Each row contains 36 spectral measurements:

```
9 pixels x 4 spectral bands = 36 predictors
```

The four spectral values for the central pixel are attributes 17, 18, 19, and 20. With the naming scheme below, those are:

```
pixel_5_band_1
pixel_5_band_2
pixel_5_band_3
pixel_5_band_4
```

```
data_url <- "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/Landsat_Landcover.csv"

landsat_raw <- read_csv(data_url, show_col_types = FALSE)

glimpse(landsat_raw)
```

```
## Rows: 6,430
## Columns: 37
## $ pixel_1_band_1 <dbl> 0.117596, -1.205362, 0.779075, 1.14656~
## $ pixel_1_band_2 <dbl> 1.241362, -1.249654, 0.148811, 0.58583~
## $ pixel_1_band_3 <dbl> 1.184036, -0.077532, 0.042617, 0.34299~
## $ pixel_1_band_4 <dbl> 0.815302, 0.444886, -0.243030, 0.02155~
## $ pixel_2_band_1 <dbl> -0.158561, -0.895959, 0.800057, 0.9475~
## $ pixel_2_band_2 <dbl> 1.256483, -0.447579, 0.164136, 0.60107~
## $ pixel_2_band_3 <dbl> 1.193546, -0.786760, 0.053370, 0.35341~
## $ pixel_2_band_4 <dbl> 0.818486, -0.554203, -0.448612, 0.0265~
## $ pixel_3_band_1 <dbl> -0.141965, -0.364672, 0.154978, 1.7881~
## $ pixel_3_band_2 <dbl> 0.879481, 0.092157, -0.345245, 1.01070~
## $ pixel_3_band_3 <dbl> 0.670010, -0.051291, -0.712483, 0.9104~
## $ pixel_3_band_4 <dbl> 0.401020, -0.178503, -0.441923, 0.4010~
## $ pixel_4_band_1 <dbl> 0.052220, -0.976988, 0.713854, 1.37548~
## $ pixel_4_band_2 <dbl> 1.204523, -0.852379, 0.504301, 0.81064~
## $ pixel_4_band_3 <dbl> 1.181239, -0.498640, -0.198662, 0.9412~
## $ pixel_4_band_4 <dbl> 0.758245, 0.124397, -0.403809, 0.23003~
## $ pixel_5_band_1 <dbl> -0.151111, -0.668189, 0.735308, 1.1046~
## $ pixel_5_band_2 <dbl> 1.214967, -0.836981, 0.341798, 1.21496~
## $ pixel_5_band_3 <dbl> 1.187378, -0.487496, -0.188412, 0.4097~
## $ pixel_5_band_4 <dbl> 0.598708, -0.084225, -0.399425, 0.2309~
## $ pixel_6_band_1 <dbl> -0.136658, -0.656748, 0.160536, 1.1264~
## $ pixel_6_band_2 <dbl> 1.011130, -0.343506, -0.343506, 1.0111~
## $ pixel_6_band_3 <dbl> 0.899623, -0.176313, -0.355636, 0.7203~
## $ pixel_6_band_4 <dbl> 0.761977, -0.234313, -0.181877, 0.2376~
## $ pixel_7_band_1 <dbl> -0.085593, -0.748307, 0.650756, 1.3871~
## $ pixel_7_band_2 <dbl> 1.211546, -0.981173, -0.016376, 0.8607~
## $ pixel_7_band_3 <dbl> 1.251179, -0.614884, -0.915862, 0.5288~
## $ pixel_7_band_4 <dbl> 0.807707, -0.192752, -0.877277, 0.2811~
## $ pixel_8_band_1 <dbl> -0.069968, -0.736996, 0.671174, 1.4123~
```

```
## $ pixel_8_band_2 <dbl> 1.219160, -0.969292, -0.006373, 1.0440~
## $ pixel_8_band_3 <dbl> 1.250463, -0.844805, -0.425752, 0.5320~
## $ pixel_8_band_4 <dbl> 0.597678, -0.400030, -0.662584, 0.2826~
## $ pixel_9_band_1 <dbl> -0.054291, -0.725852, 0.691889, 1.4380~
## $ pixel_9_band_2 <dbl> 1.233342, -0.344432, 0.356801, 1.05803~
## $ pixel_9_band_3 <dbl> 1.262255, -0.594534, -0.175259, 0.8429~
## $ pixel_9_band_4 <dbl> 0.603258, -0.183967, -0.236449, 0.1309~
## $ land_cover     <dbl> 1, 5, 7, 3, 5, 4, 5, 5, 2, 5, 4, 3, 3,~
```

```
feature_cols <- paste0(
  "pixel_",
  rep(1:9, each = 4),
  "_band_",
  rep(1:4, times = 9)
)

# The target is the last column in this CSV.
target_col <- names(landsat_raw)[ncol(landsat_raw)]

landsat <- landsat_raw %>%
  rename_with(~ feature_cols, .cols = 1:36) %>%
  rename(land_cover_code = all_of(target_col)) %>%
  mutate(
    land_cover_raw = as.character(land_cover_code),
    land_cover_clean = land_cover_raw %>%
      str_to_lower() %>%
      str_replace_all("[._-]+", " ") %>%
      str_squish(),
    land_cover_number = readr::parse_number(land_cover_raw),
    land_cover = case_when(
      land_cover_number == 1~ "red_soil",
      land_cover_number == 2 ~ "cotton_crop",
      land_cover_number == 3 ~ "grey_soil",
      land_cover_number == 4 ~ "damp_grey_soil",
      land_cover_number == 5 ~ "soil_with_vegetation_stubble",
      land_cover_number == 7 ~ "very_damp_grey_soil",
      TRUE ~ NA_character_
    ),
    land_cover = factor(land_cover)
  ) %>%
  filter(!is.na(land_cover)) %>%
  select(all_of(feature_cols), land_cover)

landsat %>% slice_head(n = 6)
```

```
## # A tibble: 6 x 37
##   pixel_1_band_1 pixel_1_band_2 pixel_1_band_3 pixel_1_band_4
##            <dbl>          <dbl>          <dbl>          <dbl>
## 1          0.118          1.24           1.18           0.815
## 2         -1.21          -1.25          -0.0775         0.445
## 3          0.779          0.149          0.0426        -0.243
## 4          1.15           0.586          0.343          0.0216
## 5         -0.764         -1.16          -0.138          0.180
## 6         -0.470         -0.725         -1.16          -1.04
## # i 33 more variables: pixel_2_band_1 <dbl>,
```

```
## #   pixel_2_band_2 <dbl>, pixel_2_band_3 <dbl>,
## #   pixel_2_band_4 <dbl>, pixel_3_band_1 <dbl>,
## #   pixel_3_band_2 <dbl>, pixel_3_band_3 <dbl>,
## #   pixel_3_band_4 <dbl>, pixel_4_band_1 <dbl>,
## #   pixel_4_band_2 <dbl>, pixel_4_band_3 <dbl>,
## #   pixel_4_band_4 <dbl>, pixel_5_band_1 <dbl>, ...
```

2. Initial Data Checks

Before modeling, check the number of rows, columns, missing values, and class balance.

```
cat("Rows and columns:\n")
```

```
## Rows and columns:
```

```
dim(landsat)
```

```
## [1] 6430   37
```

```
cat("\nMissing values:\n")
```

```
##
## Missing values:
```

```
sum(is.na(landsat))
```

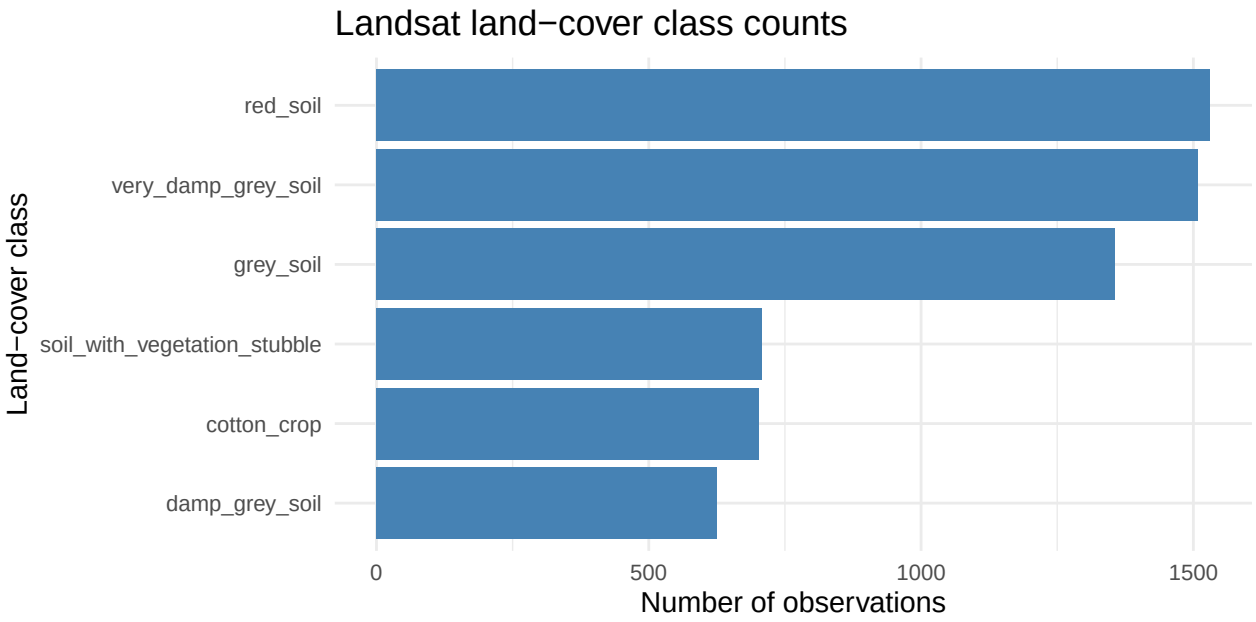
```
## [1] 0
```

```
class_counts <- landsat %>%
  count(land_cover, sort = TRUE)

class_counts
```

```
## # A tibble: 6 x 2
##   land_cover                      n
##   <fct>                       <int>
## 1 red_soil                     1531
## 2 very_damp_grey_soil          1508
## 3 grey_soil                    1356
## 4 soil_with_vegetation_stubble  707
## 5 cotton_crop                   703
## 6 damp_grey_soil                625
```

```
ggplot(class_counts, aes(x = reorder(land_cover, n), y = n)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Landsat land-cover class counts",
    x = "Land-cover class",
    y = "Number of observations"
  ) +
  theme_minimal(base_size = 13)
```



Question 1

Is this classification problem balanced? Which land-cover classes are most and least common? Most common classes are red soil, very damp grey soil, and grey soil. Least common classes are soil with vegetation stubble, cotton crop and damp grey soil. Within the most and least common groups the soil type is similar counts, but across all types it is imbalanced.

3. Why Decision Trees Have Axis-Aligned Splits

Decision trees split one feature at a time.

Examples:

```
pixel_5_band_1 <= 66
pixel_5_band_2 <= 42
pixel_5_band_4 <= 87
```

That means a decision tree creates boundaries that are vertical or horizontal in a two-feature plot. These are called **axis-aligned splits**.

This is powerful and interpretable, but it can be inefficient when the real class boundary is diagonal, curved, or depends on a smooth combination of predictors.

4. Build a Two-Feature Dataset for Visualization

To visualize decision boundaries, we will use only two predictors from the central pixel:

- `pixel_5_band_1`
- `pixel_5_band_2`

This is intentionally limited. Later, we will use all 36 spectral predictors.

```
two_feature_cols <- c("pixel_5_band_1", "pixel_5_band_2")

landsat_two <- landsat %>%
  select(all_of(two_feature_cols), land_cover) %>%
```

```
  drop_na()

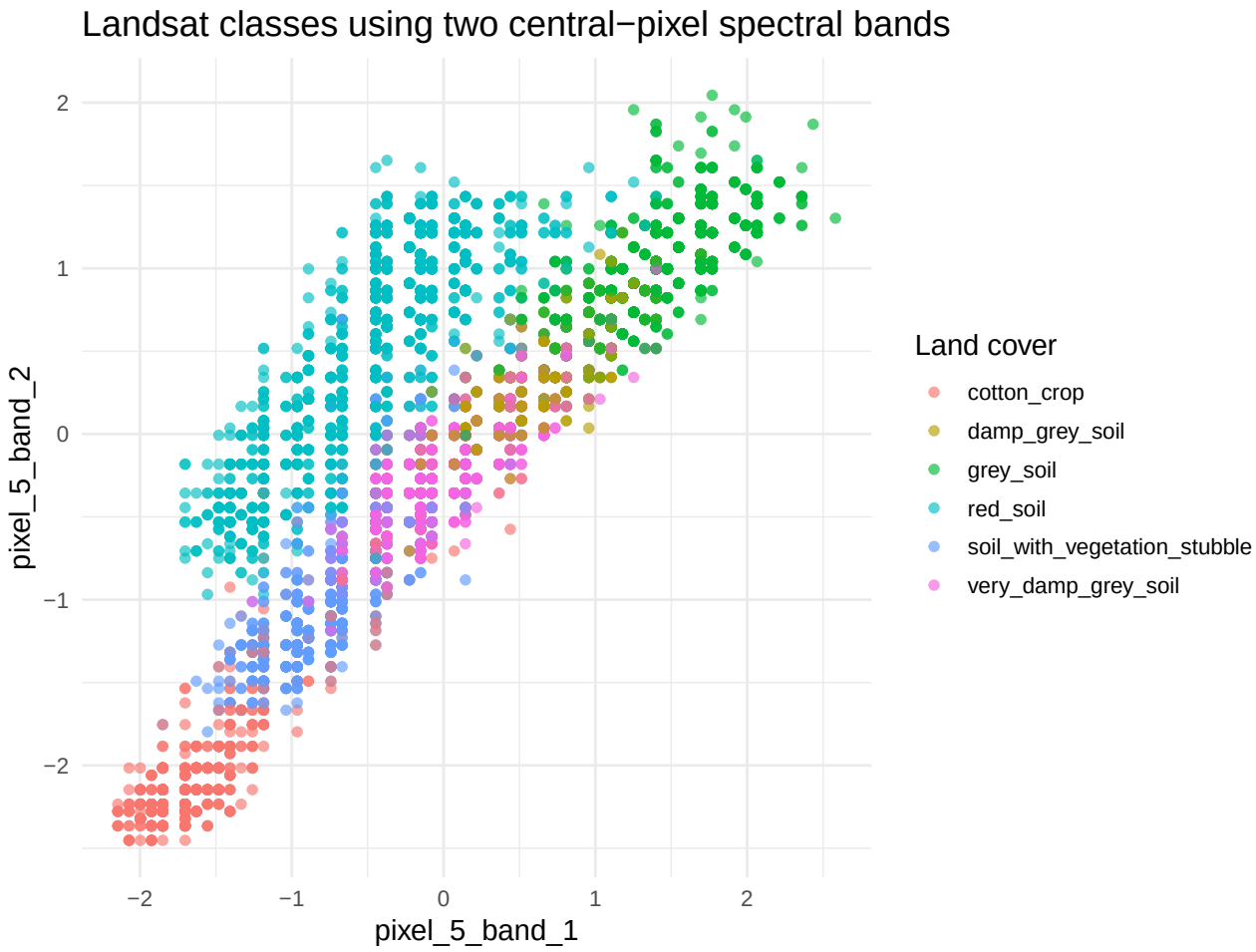
set.seed(42)
two_split <- initial_split(landsat_two, prop = 0.75, strata = land_cover)
two_train <- training(two_split)
two_test <- testing(two_split)

class_levels <- levels(landsat_two$land_cover)
class_palette <- setNames(scales::hue_pal()(length(class_levels)), class_levels)

class_palette
```

```
##                   cotton_crop                damp_grey_soil
##                     "#F8766D"                     "#B79F00"
##                     grey_soil                      red_soil
##                     "#00BA38"                     "#00BFC4"
## soil_with_vegetation_stubble           very_damp_grey_soil
##                     "#619CFF"                     "#F564E3"
```

```
ggplot(landsat_two, aes(x = pixel_5_band_1, y = pixel_5_band_2, color = land_cover)) +
  geom_point(alpha = 0.65, size = 1.6) +
  scale_color_manual(values = class_palette) +
  labs(
    title = "Landsat classes using two central-pixel spectral bands",
    x = "pixel_5_band_1",
    y = "pixel_5_band_2",
    color = "Land cover"
  ) +
  theme_minimal(base_size = 13)
```



Question 2

Looking at the scatterplot, where might a single decision tree struggle? Do the classes appear separable by only horizontal and vertical cuts?

The class boundary is largely diagonal for red soil, so a single decision tree would struggle. The other soil types have some vertical stratification, but overlap a lot horizontally and vertically.

5. Fit a Shallow Decision Tree

A shallow tree is easy to interpret, but it may be too simple.

```
shallow_tree <- rpart(
  land_cover ~ pixel_5_band_1 + pixel_5_band_2,
  data = two_train,
  method = "class",
  control = rpart.control(maxdepth = 2, cp = 0.001)
)

shallow_tree_pred_train <- predict(shallow_tree, newdata = two_train, type = "class")
shallow_tree_pred_test <- predict(shallow_tree, newdata = two_test, type = "class")

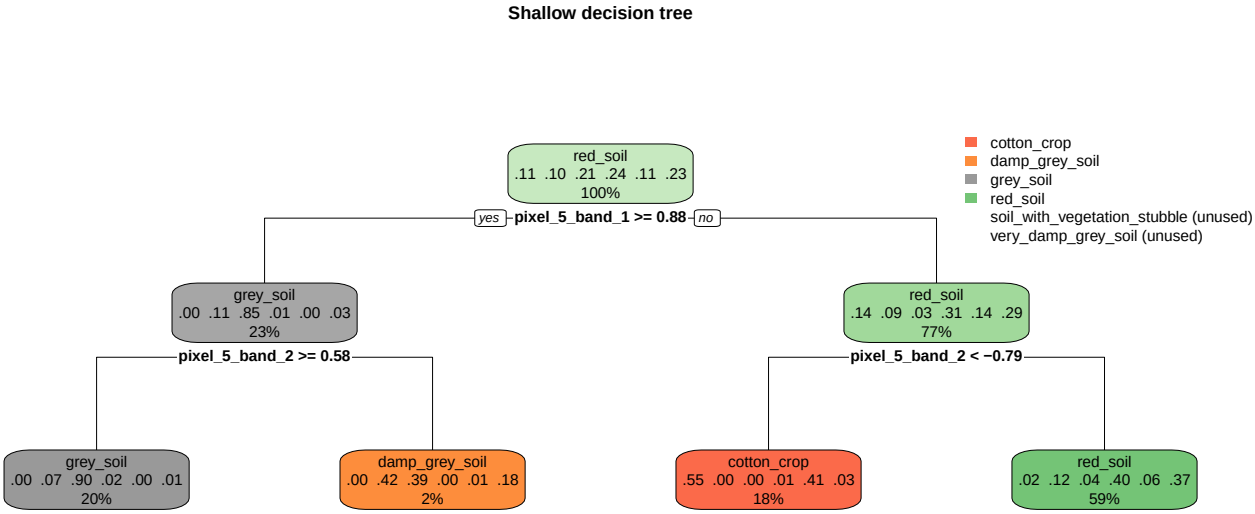
mean(shallow_tree_pred_train == two_train$land_cover)
```

```
## [1] 0.5263485
```

```
mean(shallow_tree_pred_test == two_test$land_cover)
```

```
## [1] 0.521118
```

```
rpart.plot(
  shallow_tree,
  type = 2,
  extra = 104,
  fallen.leaves = TRUE,
  main = "Shallow decision tree"
)
```



6. Decision Boundary Helper Function

The background color shows the model's predicted class across the two-feature space. The points show the observed classes. Training observations are circles and test observations are X symbols.

```
predict_tree_classes <- function(model, new_data) {
  predict(model, newdata = new_data, type = "class")
}


plot_decision_boundary <- function(model, train_data, test_data, title, predict_fun = predict_tree_class
  x_min <- min(train_data$pixel_5_band_1, test_data$pixel_5_band_1) - 5
  x_max <- max(train_data$pixel_5_band_1, test_data$pixel_5_band_1) + 5
  y_min <- min(train_data$pixel_5_band_2, test_data$pixel_5_band_2) - 5
  y_max <- max(train_data$pixel_5_band_2, test_data$pixel_5_band_2) + 5

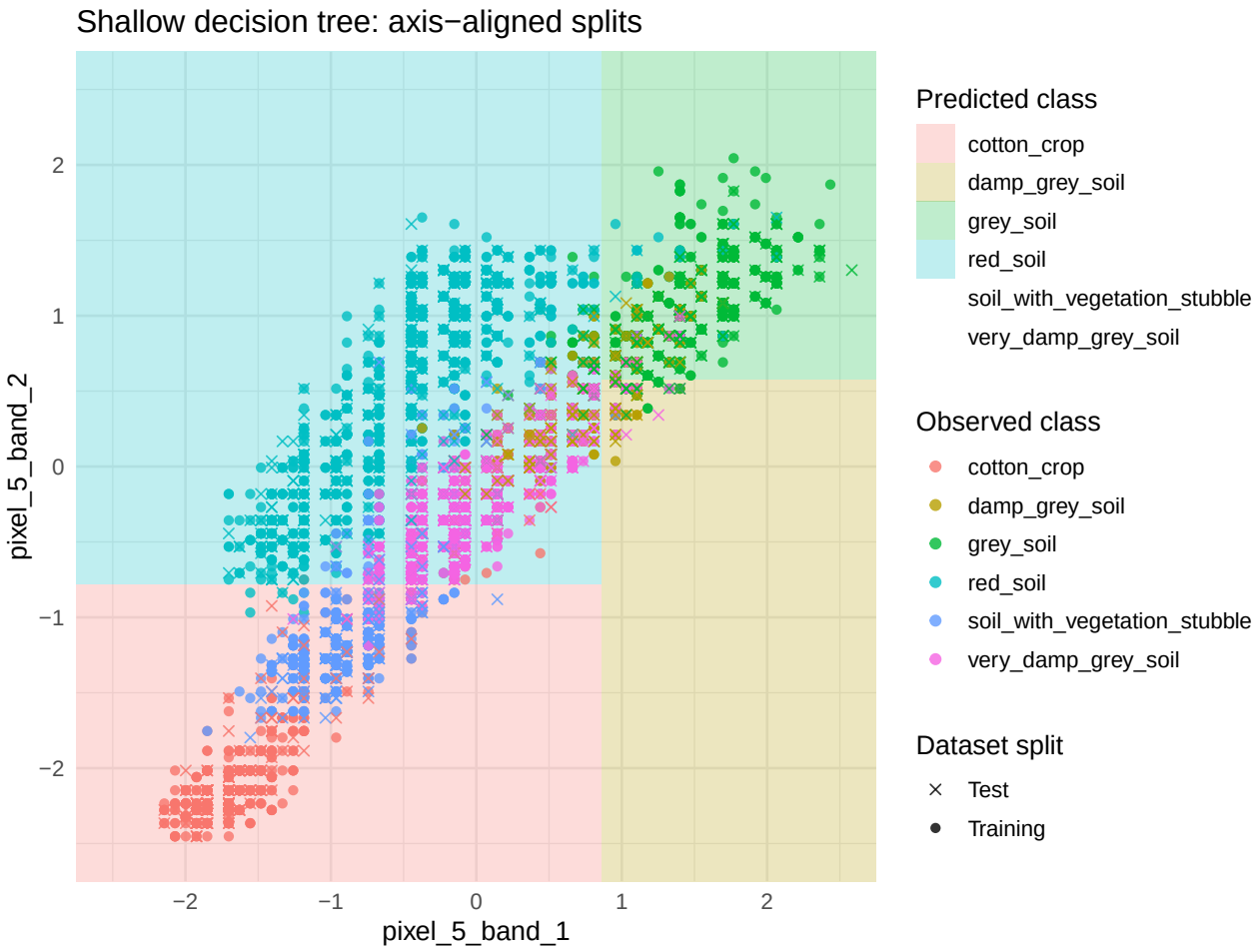
  prediction_grid <- expand_grid(
    pixel_5_band_1 = seq(x_min, x_max, length.out = 300),
    pixel_5_band_2 = seq(y_min, y_max, length.out = 300)
  )

  grid_predictions <- prediction_grid %>%
    mutate(.pred_class = predict_fun(model, prediction_grid))
```

```
  point_data <- bind_rows(
    train_data %>% mutate(split = "Training"),
    test_data %>% mutate(split = "Test")
  )

  ggplot() +
    geom_raster(
      data = grid_predictions,
      aes(x = pixel_5_band_1, y = pixel_5_band_2, fill = .pred_class),
      alpha = 0.25
    ) +
    geom_point(
      data = point_data,
      aes(x = pixel_5_band_1, y = pixel_5_band_2, color = land_cover, shape = split),
      size = 1.8,
      alpha = 0.8
    ) +
    scale_fill_manual(values = class_palette, drop = FALSE) +
    scale_color_manual(values = class_palette, drop = FALSE) +
    scale_shape_manual(values = c("Training" = 16, "Test" = 4)) +
    coord_cartesian(xlim = c(-2.5, 2.5), ylim = c(-2.5, 2.5)) +
    labs(
      title = title,
      x = "pixel_5_band_1",
      y = "pixel_5_band_2",
      fill = "Predicted class",
      color = "Observed class",
      shape = "Dataset split"
    ) +
    theme_minimal(base_size = 12)
}
```

```
plot_decision_boundary(
  shallow_tree,
  two_train,
  two_test,
  "Shallow decision tree: axis-aligned splits"
)
```



Question 3

What do you notice about the decision boundary? Why does it look like rectangles or stair steps? The decision boundary looks like rectangles, and there are no decision boundaries for soil with vegetation stubble and very damp grey soil.

7. Compare Single Tree Depths

Deeper trees are more flexible, but they can overfit.

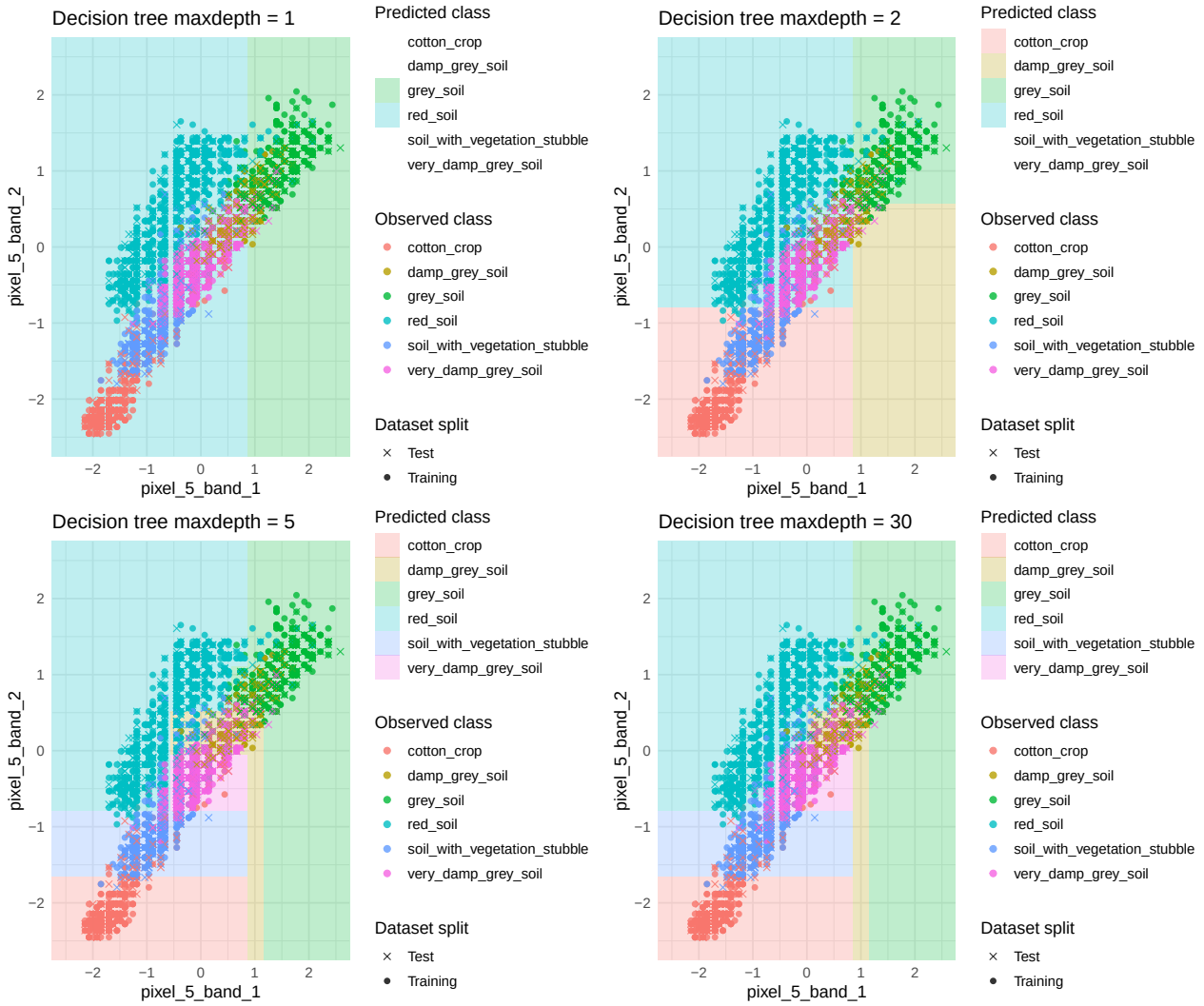
```
tree_depths <- c(1, 2, 5, 30)

tree_depth_plots <- map(
  tree_depths,
  function(depth) {
    tree_fit <- rpart(
      land_cover ~ pixel_5_band_1 + pixel_5_band_2,
      data = two_train,
      method = "class",
      control = rpart.control(maxdepth = depth, cp = 0.001)
    )

    plot_decision_boundary(
```

```
      tree_fit,
      two_train,
      two_test,
      paste0("Decision tree maxdepth = ", depth)
    )
  }
)

wrap_plots(tree_depth_plots, ncol = 2)
```



Question 4

How does the decision boundary change as the tree gets deeper? Which model seems most likely to overfit? There are decision boundaries created for all observed classes. There is not much change from 5 to 30. A decsion tree of 30 seems most likely to overfit.

8. Compare Tree-Based Ensembles on the Same Two Features

Now compare:

- Decision tree
- Random forest
- XGBoost

This section is still using only two features so the boundaries can be visualized.

```
# Decision tree
boundary_tree <- rpart(
  land_cover ~ pixel_5_band_1 + pixel_5_band_2,
  data = two_train,
  method = "class",
  control = rpart.control(maxdepth = 5, cp = 0.001)
)

# Random forest
boundary_rf <- ranger(
  land_cover ~ pixel_5_band_1 + pixel_5_band_2,
  data = two_train,
  num.trees = 200,
  max.depth = 6,
  probability = FALSE,
  seed = 42
)


# XGBoost through tidymodels
boundary_xgb_spec <- boost_tree(
  trees = 100,
  tree_depth = 2,
  learn_rate = 0.1,
  sample_size = 0.9,
  mtry = 2
) %>%
  set_mode("classification") %>%
  set_engine("xgboost", eval_metric = "mlogloss")

boundary_xgb_wf <- workflow() %>%
  add_formula(land_cover ~ pixel_5_band_1 + pixel_5_band_2) %>%
  add_model(boundary_xgb_spec)

boundary_xgb <- fit(boundary_xgb_wf, data = two_train)
```

```
predict_ranger_classes <- function(model, new_data) {
  predict(model, data = new_data)$predictions %>%
    factor(levels = class_levels)
}

predict_workflow_classes <- function(model, new_data) {
  predict(model, new_data, type = "class")$.pred_class %>%
    factor(levels = class_levels)
}

two_feature_accuracy <- tibble(
  model = c("Decision tree", "Random forest", "XGBoost"),
  train_accuracy = c(
```

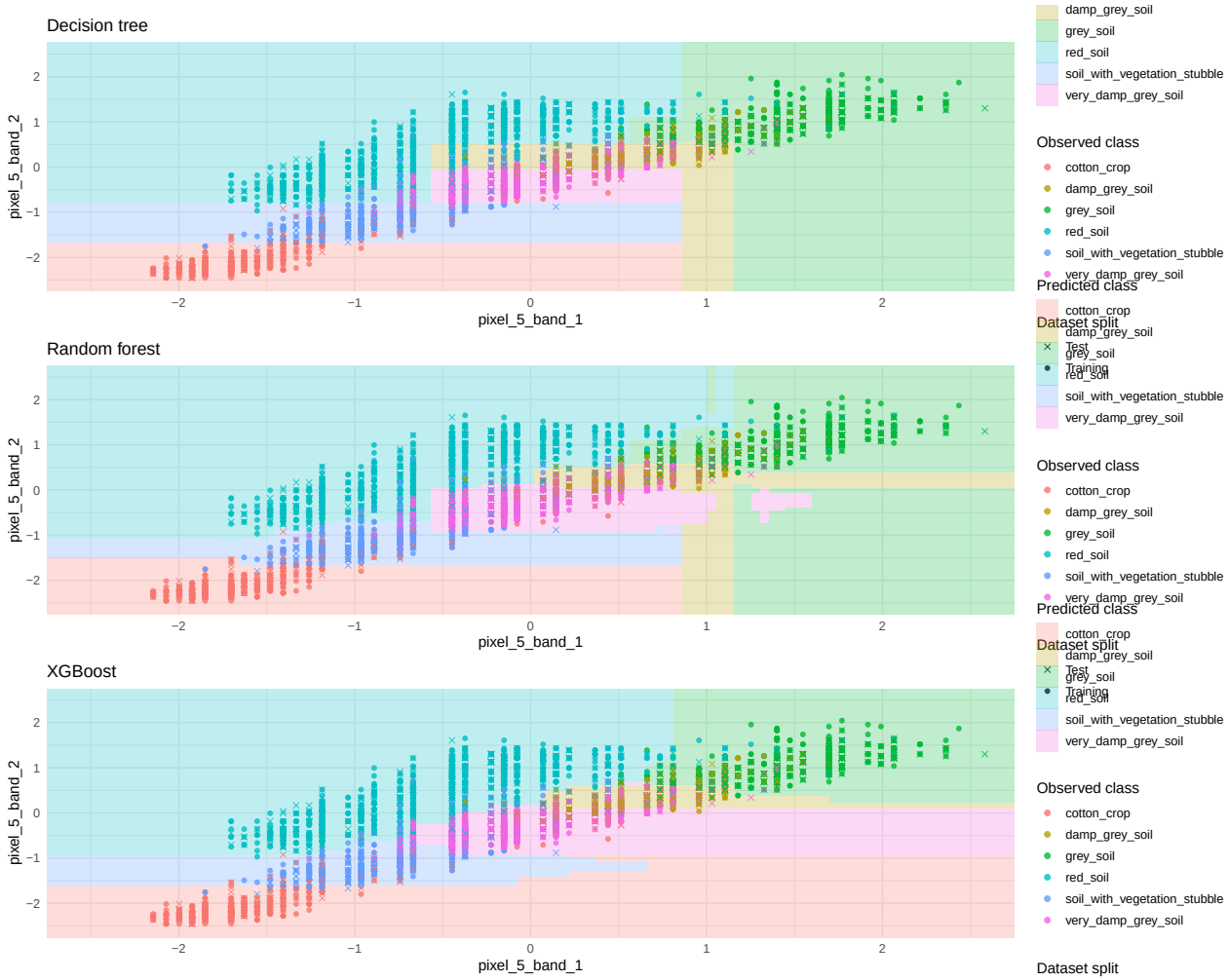
```
    mean(predict_tree_classes(boundary_tree, two_train) == two_train$land_cover),
    mean(predict_ranger_classes(boundary_rf, two_train) == two_train$land_cover),
    mean(predict_workflow_classes(boundary_xgb, two_train) == two_train$land_cover)
  ),
  test_accuracy = c(
    mean(predict_tree_classes(boundary_tree, two_test) == two_test$land_cover),
    mean(predict_ranger_classes(boundary_rf, two_test) == two_test$land_cover),
    mean(predict_workflow_classes(boundary_xgb, two_test) == two_test$land_cover)
  )
)

two_feature_accuracy
```

```
## # A tibble: 3 x 3
##   model         train_accuracy test_accuracy
##   <chr>                  <dbl>         <dbl>
## 1 Decision tree          0.801         0.786
## 2 Random forest          0.832         0.815
## 3 XGBoost                0.830         0.816
```

```
p1 <- plot_decision_boundary(boundary_tree, two_train, two_test, "Decision tree")
p2 <- plot_decision_boundary(boundary_rf, two_train, two_test, "Random forest", predict_fun = predict_ra
p3 <- plot_decision_boundary(boundary_xgb, two_train, two_test, "XGBoost", predict_fun = predict_workfl

p1 / p2 / p3
```



Question 5

Compare the three decision boundaries.

1. Which model has the simplest boundary? The model with the least number of edges is the decision tree.
2. Which model has the most complex boundary? A random forest has the most complex boundaries because there are gaps in the boundary.
3. Which model seems most likely to overfit? The random forest seems overfit. The complex boundaries do not seem that they would fit new data well.
4. Why do all tree-based boundaries still look somewhat blocky? We are only using two features to create boundaries.

Part 2: Best Practices With All Spectral Features

The two-feature plots are useful for learning, but they are not the best possible models. In real modeling, we should use a proper train/test split, cross-validation, preprocessing, and tuning.

We will now use all 36 spectral predictors.

Best-practice reminders:

- Keep the test set untouched until final evaluation.
- Use cross-validation for model comparison and tuning.

- Use workflows or consistent preprocessing to avoid leakage.
- Tune complexity controls such as tree depth, number of trees, and learning rate.
- Compare against a simple baseline.

9. Prepare the Full Feature Dataset

```
set.seed(42)
landsat_split <- initial_split(landsat, prop = 0.75, strata = land_cover)
train_data <- training(landsat_split)
test_data <- testing(landsat_split)

nrow(train_data)
```

```
## [1] 4820
```

```
nrow(test_data)
```

```
## [1] 1610
```

```
ncol(train_data) - 1
```

```
## [1] 36
```

10. Define Baseline and Candidate Models

Tree models do not require feature scaling. We include a median imputer in the recipe as a best-practice safeguard, although this dataset should not have missing predictor values.

```
landsat_recipe <- recipe(land_cover ~ ., data = train_data) %>%
  step_impute_median(all_numeric_predictors())

baseline_spec <- null_model() %>%
  set_mode("classification") %>%
  set_engine("parsnip")

tree_spec <- decision_tree(
  tree_depth = 8,
  min_n = 5,
  cost_complexity = 0.001
) %>%
  set_mode("classification") %>%
  set_engine("rpart")

rf_spec <- rand_forest(
  trees = 300,
  mtry = 6,
  min_n = 2
) %>%
  set_mode("classification") %>%
  set_engine("ranger")

xgb_spec <- boost_tree(
  trees = 150,
  tree_depth = 2,
  learn_rate = 0.05,
```

```
  sample_size = 0.9,
  mtry = 24
) %>%
  set_mode("classification") %>%
  set_engine("xgboost", eval_metric = "mlogloss")

make_workflow <- function(model_spec) {
  workflow() %>%
    add_recipe(landsat_recipe) %>%
    add_model(model_spec)
}

candidate_workflows <- list(
  "Baseline" = make_workflow(baseline_spec),
  "Decision tree" = make_workflow(tree_spec),
  "Random forest" = make_workflow(rf_spec),
  "XGBoost" = make_workflow(xgb_spec)
)
```

11. Cross-Validation Comparison

Use cross-validation on the training set to compare models before touching the test set.

```
set.seed(42)
cv_folds <- vfold_cv(train_data, v = 5, strata = land_cover)

classification_metrics <- metric_set(accuracy, f_meas)

cv_results <- imap_dfr(
  candidate_workflows,
  function(wf, model_name) {
    fit_resamples(
      wf,
      resamples = cv_folds,
      metrics = classification_metrics,
      control = control_resamples(save_pred = TRUE)
    ) %>%
      collect_metrics() %>%
      mutate(model = model_name)
  }
)
```

```
## > A | warning: While computing multiclass `precision()`, some levels had no
##                predicted events (i.e. `true_positive + false_positive = 0`).
##                Precision is undefined in this case, and those levels will be
##                removed from the averaged result.
##                Note that the following number of true events actually
##                occurred for each problematic event level:
##                'cotton_crop': 108, 'damp_grey_soil': 88, 'grey_soil': 203,
##                'soil_with_vegetation_stubble': 108, 'very_damp_grey_soil':
##                225
```

```
## There were issues with some computations   A: x1                                                > B
##                predicted events (i.e. `true_positive + false_positive = 0`).
```

```
##                Precision is undefined in this case, and those levels will be
##                removed from the averaged result.
##                Note that the following number of true events actually
##                occurred for each problematic event level:
##                'cotton_crop': 113, 'damp_grey_soil': 95, 'grey_soil': 200,
##                'soil_with_vegetation_stubble': 106, 'very_damp_grey_soil':
##                221
## There were issues with some computations   A: x1There were issues with some computations   A: x1   B
##                predicted events (i.e. `true_positive + false_positive = 0`).
##                Precision is undefined in this case, and those levels will be
##                removed from the averaged result.
##                Note that the following number of true events actually
##                occurred for each problematic event level:
##                'cotton_crop': 102, 'damp_grey_soil': 98, 'grey_soil': 205,
##                'soil_with_vegetation_stubble': 105, 'very_damp_grey_soil':
##                225
## There were issues with some computations   A: x1   B: x1
##                predicted events (i.e. `true_positive + false_positive = 0`).
##                Precision is undefined in this case, and those levels will be
##                removed from the averaged result.
##                Note that the following number of true events actually
##                occurred for each problematic event level:
##                'cotton_crop': 102, 'damp_grey_soil': 91, 'grey_soil': 208,
##                'soil_with_vegetation_stubble': 110, 'very_damp_grey_soil':
##                224
## There were issues with some computations   A: x1   B: x1
##                predicted events (i.e. `true_positive + false_positive = 0`).
##                Precision is undefined in this case, and those levels will be
##                removed from the averaged result.
##                Note that the following number of true events actually
##                occurred for each problematic event level:
##                'cotton_crop': 97, 'damp_grey_soil': 91, 'grey_soil': 207,
##                'soil_with_vegetation_stubble': 99, 'very_damp_grey_soil': 233
## There were issues with some computations   A: x1   B: x1There were issues with some computations   A
```

```
cv_results
```

```
## # A tibble: 8 x 7
##   .metric  .estimator  mean     n std_err .config         model
##   <chr>    <chr>      <dbl> <int>   <dbl> <chr>           <chr>
## 1 accuracy multiclass 0.240     5 0.00129 pre0_mod0_post0 Base~
## 2 f_meas   macro      0.387     5 0.00167 pre0_mod0_post0 Base~
## 3 accuracy multiclass 0.856     5 0.00547 pre0_mod0_post0 Deci~
## 4 f_meas   macro      0.820     5 0.00820 pre0_mod0_post0 Deci~
## 5 accuracy multiclass 0.910     5 0.00359 pre0_mod0_post0 Rand~
## 6 f_meas   macro      0.889     5 0.00518 pre0_mod0_post0 Rand~
## 7 accuracy multiclass 0.876     5 0.00314 pre0_mod0_post0 XGBo~
## 8 f_meas   macro      0.850     5 0.00438 pre0_mod0_post0 XGBo~
```

```
cv_summary <- cv_results %>%
  select(model, .metric, mean, std_err) %>%
  arrange(.metric, desc(mean))

cv_summary
```

```
## # A tibble: 8 x 4
##   model         .metric   mean std_err
##   <chr>         <chr>    <dbl>   <dbl>
## 1 Random forest accuracy 0.910 0.00359
## 2 XGBoost       accuracy 0.876 0.00314
## 3 Decision tree accuracy 0.856 0.00547
## 4 Baseline      accuracy 0.240 0.00129
## 5 Random forest f_meas   0.889 0.00518
## 6 XGBoost       f_meas   0.850 0.00438
## 7 Decision tree f_meas   0.820 0.00820
## 8 Baseline      f_meas   0.387 0.00167
```

Question 6

Which model has the best cross-validated macro F1 score? Does any model have much higher training accuracy than validation accuracy? What would that suggest? The random forest has the best f1 score. The baseline model is the only model with an f1 score more than a few percent higher than the accuracy, which typically suggests overfitting. However, this is a baseline null model with low f1 and low accuracy.

12. Tune XGBoost

XGBoost has many tuning parameters. We will tune only a few so the activity stays manageable.

Important parameters:

- `tree_depth`: tree complexity
- `learn_rate`: contribution of each tree
- `trees`: number of boosting rounds
- `sample_size`: fraction of rows used per tree
- `mtry`: number of predictors sampled at each split

Best practice: tune on the training set using cross-validation, then evaluate once on the held-out test set.

```
xgb_tune_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  sample_size = tune(),
  mtry = tune(),
  min_n = 2
) %>%
  set_mode("classification") %>%
  set_engine("xgboost", eval_metric = "mlogloss")

xgb_tune_wf <- workflow() %>%
  add_recipe(landsat_recipe) %>%
  add_model(xgb_tune_spec)

xgb_grid <- crossing(
  trees = c(150, 300),
  tree_depth = c(2, 4),
  learn_rate = c(0.03, 0.1),
  sample_size = c(0.8, 1.0),
  mtry = c(12, 24)
)
```

```r
set.seed(42)
xgb_tuned <- tune_grid(
  xgb_tune_wf,
  resamples = cv_folds,
  grid = xgb_grid,
  metrics = classification_metrics,
  control = control_grid(save_pred = TRUE)
)

show_best(xgb_tuned, metric = "f_meas")
```

```
## # A tibble: 5 x 11
##    mtry trees tree_depth learn_rate sample_size .metric
##   <dbl> <dbl>      <dbl>      <dbl>       <dbl> <chr>
## 1    24   300          4        0.1         0.8 f_meas
## 2    12   300          4        0.1         0.8 f_meas
## 3    12   300          4        0.1         1   f_meas
## 4    24   300          4        0.1         1   f_meas
## 5    12   150          4        0.1         0.8 f_meas
## # i 5 more variables: .estimator <chr>, mean <dbl>, n <int>,
## #   std_err <dbl>, .config <chr>
```

```r
best_xgb <- select_best(xgb_tuned, metric = "f_meas")
best_xgb
```

```
## # A tibble: 1 x 6
##    mtry trees tree_depth learn_rate sample_size .config
##   <dbl> <dbl>      <dbl>      <dbl>       <dbl> <chr>
## 1    24   300          4        0.1         0.8 pre0_mod31_pos~
```

```r
final_xgb_wf <- finalize_workflow(xgb_tune_wf, best_xgb)
```

Question 7

Which parameters were selected for XGBoost? Were the selected trees shallow or deep? What does `sample_size` do? - `tree_depth`: 4 - `learn_rate`: 0.1 - `trees`: 300 - `sample_size`: 0.8 - `mtry`: 24 The selected trees were deep, and smaple size is the fraction of rows used per tree, where lower values prevent overfitting.

13. Final Test-Set Evaluation

Now evaluate the selected models on the test set.

The test set should be used only after model comparison and tuning decisions are complete.

```r
final_workflows <- list(
  "Decision tree" = candidate_workflows[["Decision tree"]],
  "Random forest" = candidate_workflows[["Random forest"]],
  "Tuned XGBoost" = final_xgb_wf
)

final_predictions <- imap_dfr(
  final_workflows,
  function(wf, model_name) {
    fit(wf, data = train_data) %>%
```

```
      predict(test_data, type = "class") %>%
      bind_cols(test_data %>% select(land_cover)) %>%
      mutate(model = model_name)
  }
)

final_predictions %>%
  group_by(model) %>%
  accuracy(truth = land_cover, estimate = .pred_class)
```

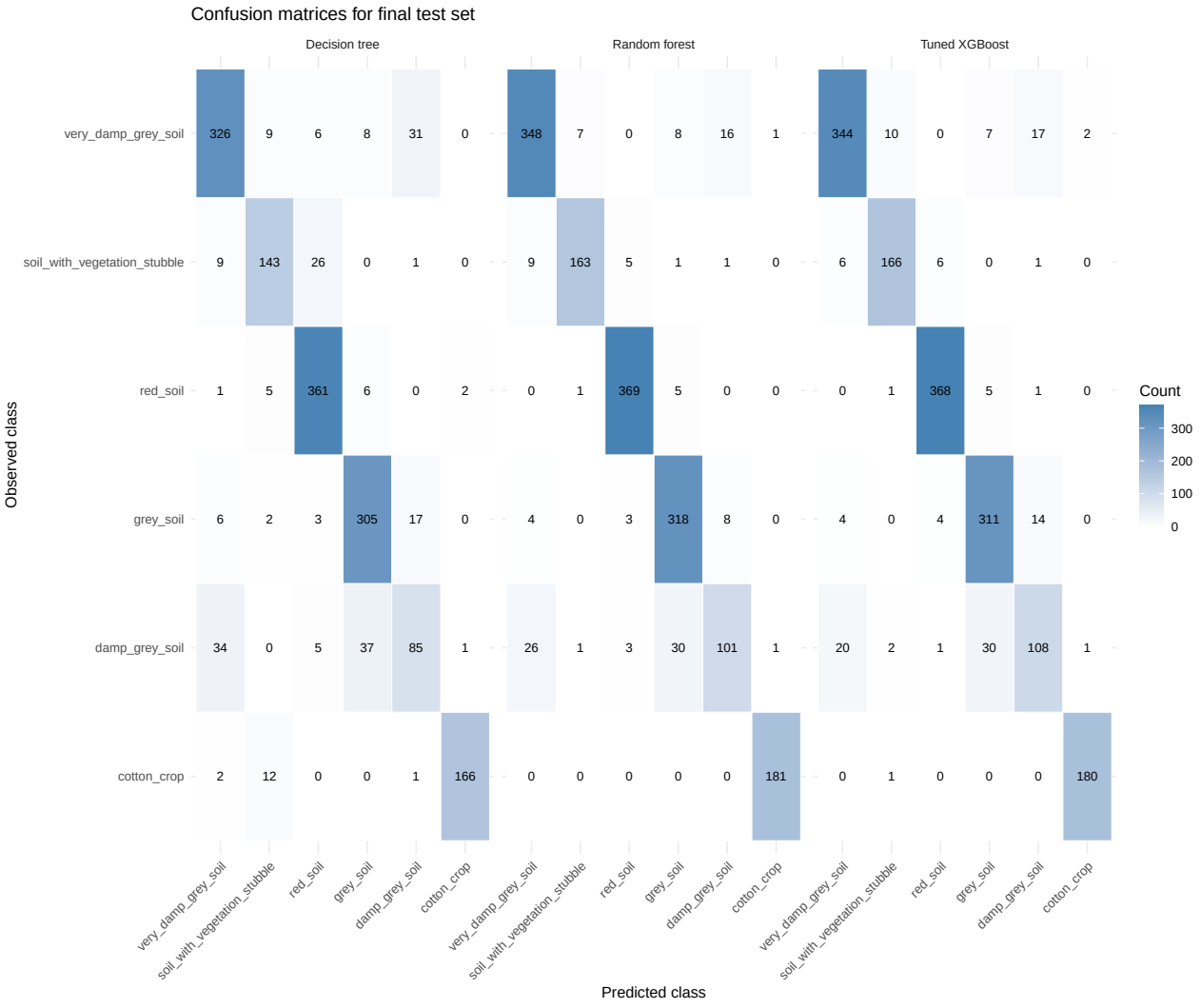
```
## # A tibble: 3 x 4
##   model         .metric  .estimator .estimate
##   <chr>         <chr>    <chr>          <dbl>
## 1 Decision tree accuracy multiclass     0.861
## 2 Random forest accuracy multiclass     0.919
## 3 Tuned XGBoost accuracy multiclass     0.917
```

```
final_predictions %>%
  group_by(model) %>%
  f_meas(truth = land_cover, estimate = .pred_class)
```

```
## # A tibble: 3 x 4
##   model         .metric .estimator .estimate
##   <chr>         <chr>   <chr>          <dbl>
## 1 Decision tree f_meas  macro          0.836
## 2 Random forest f_meas  macro          0.904
## 3 Tuned XGBoost f_meas  macro          0.904
```

```
confusion_plot_data <- final_predictions %>%
  count(model, land_cover, .pred_class, name = "n") %>%
  complete(model, land_cover, .pred_class, fill = list(n = 0))

ggplot(confusion_plot_data, aes(x = .pred_class, y = land_cover, fill = n)) +
  geom_tile(color = "white") +
  geom_text(aes(label = n), size = 3) +
  facet_wrap(~ model) +
  scale_x_discrete(limits = rev) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(
    title = "Confusion matrices for final test set",
    x = "Predicted class",
    y = "Observed class",
    fill = "Count"
  ) +
  theme_minimal(base_size = 11) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Question 8

Which model performs best on the test set? Is that the same model that performed best during cross-validation? If not, why might that happen? The random forest and xgboost perform the best on the test set. The random forest performed best during cross-validation. Changes in performance from cross-validation to testing could be due to over and underfitting of the model, depending on the variance of the data in the test set.

14. Feature Importance and Permutation Importance

Tree-based models often provide built-in feature importance, but those values can be biased. Permutation importance is usually easier to explain:

> A feature is important if model performance gets worse when that feature is randomly shuffled.

Here we will use permutation importance on the tuned XGBoost model.

```
final_xgb_fit <- fit(final_xgb_wf, data = train_data)

baseline_predictions <- predict(final_xgb_fit, test_data, type = "class") %>%
  bind_cols(test_data %>% select(land_cover))
```

```r
baseline_f1 <- f_meas(
  baseline_predictions,
  truth = land_cover,
  estimate = .pred_class
) %>%
  pull(.estimate)

set.seed(42)
permute_one_feature <- function(feature_name, repeats = 10) {
  map_dfr(seq_len(repeats), function(rep_id) {
    permuted_test <- test_data
    permuted_test[[feature_name]] <- sample(permuted_test[[feature_name]])

    permuted_predictions <- predict(final_xgb_fit, permuted_test, type = "class") %>%
      bind_cols(test_data %>% select(land_cover))

    permuted_f1 <- f_meas(
      permuted_predictions,
      truth = land_cover,
      estimate = .pred_class
    ) %>%
      pull(.estimate)

    tibble(
      feature = feature_name,
      repeat_id = rep_id,
      importance = baseline_f1 - permuted_f1
    )
  })
}

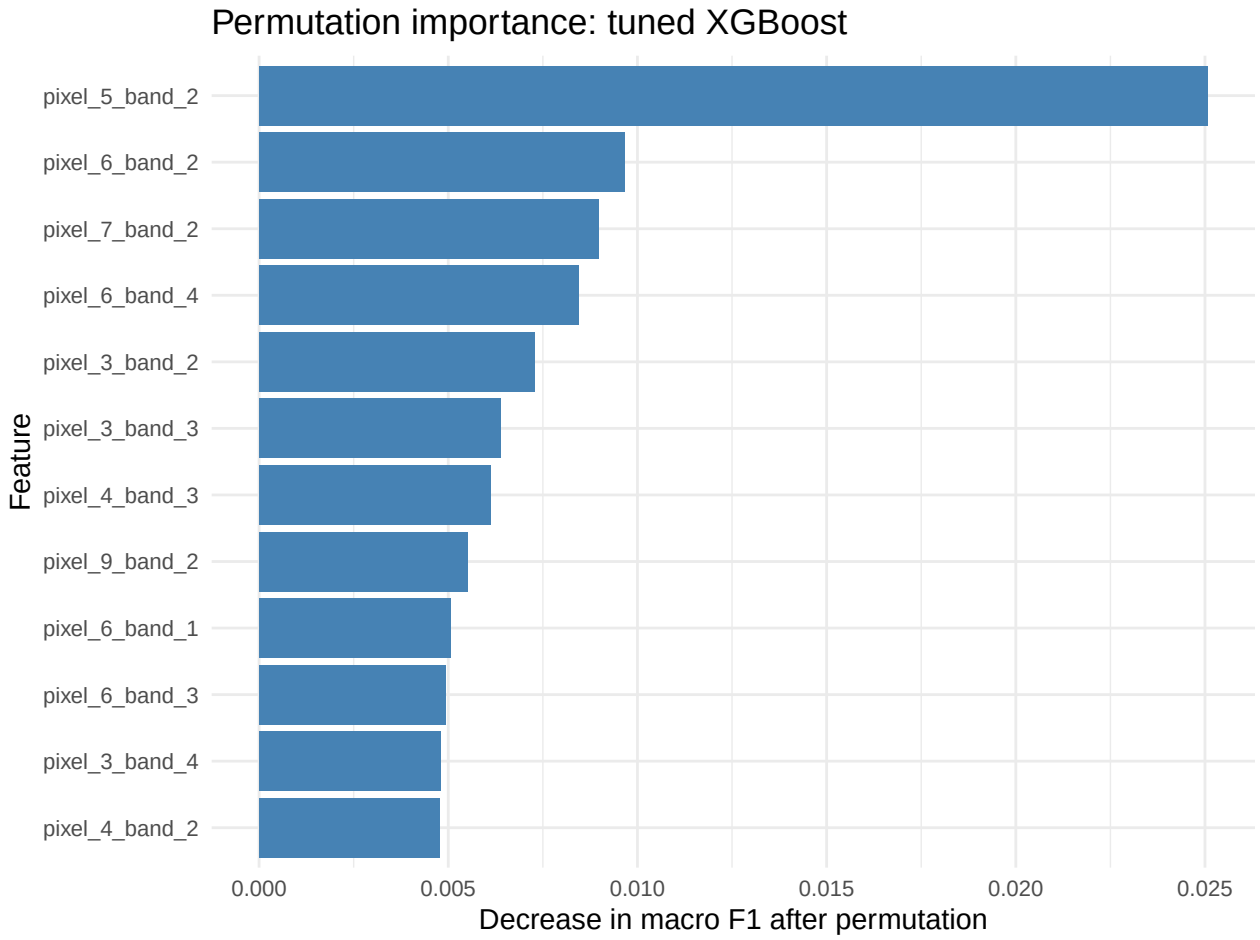
importance_df <- map_dfr(feature_cols, permute_one_feature) %>%
  group_by(feature) %>%
  summarize(
    importance_mean = mean(importance),
    importance_sd = sd(importance),
    .groups = "drop"
  ) %>%
  arrange(desc(importance_mean))

importance_df %>% slice_head(n = 12)
```

```
## # A tibble: 12 x 3
##    feature        importance_mean importance_sd
##    <chr>                    <dbl>         <dbl>
##  1 pixel_5_band_2         0.0251        0.00419
##  2 pixel_6_band_2         0.00965       0.00315
##  3 pixel_7_band_2         0.00899       0.00345
##  4 pixel_6_band_4         0.00846       0.00178
##  5 pixel_3_band_2         0.00727       0.00324
##  6 pixel_3_band_3         0.00639       0.00267
##  7 pixel_4_band_3         0.00612       0.00250
##  8 pixel_9_band_2         0.00550       0.00276
```

```
##  9 pixel_6_band_1         0.00505       0.00381
## 10 pixel_6_band_3         0.00492       0.00243
## 11 pixel_3_band_4         0.00481       0.00323
## 12 pixel_4_band_2         0.00478       0.00298
```

```r
importance_df %>%
  slice_max(importance_mean, n = 12) %>%
  ggplot(aes(x = reorder(feature, importance_mean), y = importance_mean)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Permutation importance: tuned XGBoost",
    x = "Feature",
    y = "Decrease in macro F1 after permutation"
  ) +
  theme_minimal(base_size = 13)
```



Question 9

Which spectral features appear most important? Do the central-pixel bands appear useful? Why might neighboring-pixel spectral values also help classify land cover? Band 2, pixels 5,6,7 appear to be the most important. Only pixel 5's band 2 seems to be important, and band 2 itself seems to have a larger influence on importance in the model overall. I would expect neighboring pixels to be more similar to the central pixel's land cover, opposed to having land cover be randomly distributed.

Be careful: feature importance describes what the model used. It does not prove causation.

15. Best-Practice Reflection

Answer these questions in a few sentences each.

1. Why is a single decision tree easy to interpret but prone to overfitting? A single decision tree is easy to interpret because it is using binary predictors to classify at each split, which makes the flow chart easy to follow. Depending on the number of splits allowed it can keep splitting and splitting on fine differences, which would result in overfitting.
2. Why do decision tree boundaries look blocky in two dimensions? If a boundary is only visualized in the x and y axis, it will have to draw vertical and horizontal lines to show the boundary. Thus creating a blocky, boundary.
3. How is a random forest different from a single tree? A random forest creates many trees, and then uses the majority classification from all of those trees to determine the outcome. This would reduce error that could be propagated throughout the model in a single tree.
4. How is boosting different from random forest bagging? Boosting builds one model at a time, and improves upon that model. It can use multiple weak models to be informative predictors.
5. Why should XGBoost be tuned with cross-validation instead of directly on the test set? It would be considered data leakage to tune a model with the test set. Especially a boosted model, where the tuning is happening sequentially, so the final model would be heavily informed by test data if it is used in tuning.
6. Why is permutation importance useful, and what are its limitations? Permutation importance is useful for interpreting model results, and knowing what features are driving the model outcomes. A limitation is that it cannot show causality, it is only showing the importance as a predictor in the model.
7. For environmental remote-sensing datasets, why might random train/test splits still be too optimistic?

There is a lot of spatial autocorrelation in geoscience datasets, so neghboring cells may play a big role in how the model is informed and contribute to data leakage. Spatial blocking could be a more unbiased way to split the data.

Key Takeaways

- A decision tree makes one-feature-at-a-time splits, creating axis-aligned boundaries.
- A shallow tree may underfit; a deep tree may overfit.
- Random forests average many trees to reduce variance.
- XGBoost builds trees sequentially so later trees correct earlier mistakes.
- XGBoost is powerful, but tuning and validation matter.
- Workflows help prevent preprocessing leakage.
- Feature importance helps explain model behavior, but importance is not causality.
- In environmental science, spatial and temporal validation may matter more than small differences among algorithms.